```
                    ReadOnly="True" SortExpression="Order ID" />
                <asp:BoundField DataField="Price" HeaderText="Price"
SortExpression="Price" />
                <asp:BoundField DataField="Game Name" HeaderText="Game Name"
SortExpression="Game Name" />
            </Columns>
            <AlternatingRowStyle BackColor="Lime" BorderColor="Lime"
ForeColor="Black" />
        </asp:GridView>
        <br />
        <asp:SqlDataSource ID="SqlDataSource1" runat="server"
DataSourceMode="DataReader" ConnectionString="<%$
ConnectionStrings:ConnectionString %>"
            DeleteCommand="DELETE FROM [Table3] WHERE [Order ID] = @Order_ID"
InsertCommand="INSERT INTO [Table3] ([User ID], [Price], [Game Name]) VALUES
(@User_ID, @Price, @Game_Name)"
            SelectCommand="SELECT * FROM [Table3]" UpdateCommand="UPDATE
[Table3] SET [User ID] = @User_ID, [Price] = @Price, [Game Name] = @Game_Name
WHERE [Order ID] = @Order_ID">
            <DeleteParameters>
                <asp:Parameter Name="Order_ID" Type="Int32" />
            </DeleteParameters>
            <UpdateParameters>
                <asp:Parameter Name="User_ID" Type="String" />
                <asp:Parameter Name="Price" Type="Decimal" />
                <asp:Parameter Name="Game_Name" Type="String" />
                <asp:Parameter Name="Order_ID" Type="Int32" />
            </UpdateParameters>
            <InsertParameters>
                <asp:Parameter Name="User_ID" Type="String" />
                <asp:Parameter Name="Price" Type="Decimal" />
                <asp:Parameter Name="Game_Name" Type="String" />
            </InsertParameters>
        </asp:SqlDataSource>
        <br />
        <asp:Label ID="Label1" runat="server" ForeColor="Lime"
Text="Calculate your Session Total Below"></asp:Label><br />
        <br />
        <asp:Button ID="Button1" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
            Text="Calculate" />
        <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox></div>
    </form>
</body>
</html>
```

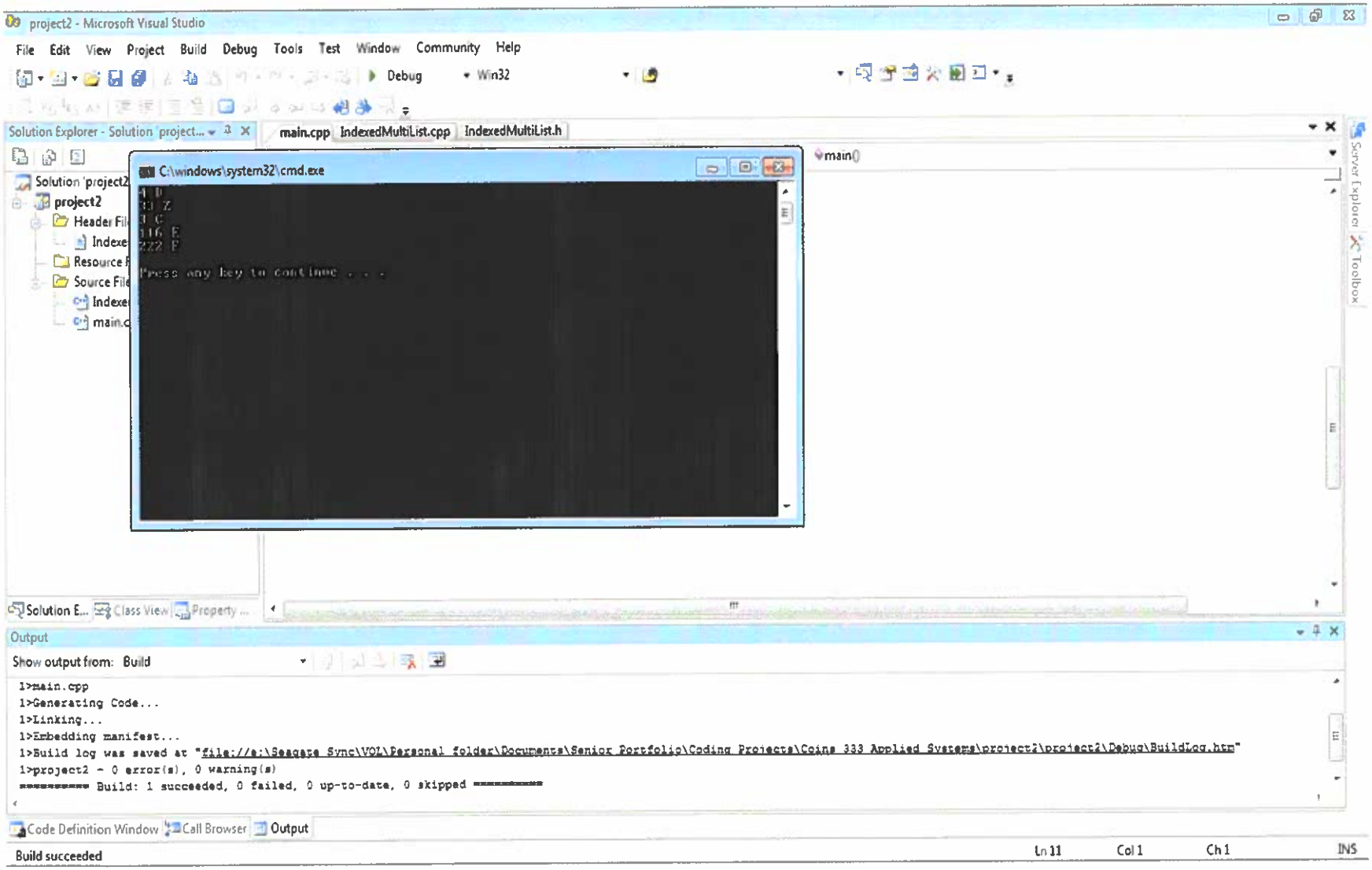
```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="userDefault.aspx.vb"
Inherits="Users_userDefault" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
    <title>Untitled Page</title>
</head>
<body bgcolor="black">
    <form id="form1" runat="server">
    <div>
         <div style="float: left; width: 112px; height: 464px">
            <asp:Button ID="Button2" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Login" /> 
            <asp:Button ID="Button1" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="CreateAccount" Width="99px" /><br /><asp:Button
ID="Button6" runat="server" BackColor="Black" BorderColor="Lime"
ForeColor="Lime"
                Text="Logout" Width="101px" /><br />
            <br />
                <asp:Label ID="Label1" runat="server"
BackColor="Black" BorderColor="Green"
                BorderStyle="Groove" ForeColor="Yellow"
Text="Platform:"></asp:Label>
            <asp:DropDownList ID="DropDownList1" runat="server"
BackColor="Lime" ForeColor="Black">
                <asp:ListItem>Xbox 360</asp:ListItem>
                <asp:ListItem>PS3</asp:ListItem>
                <asp:ListItem>Wii</asp:ListItem>
                <asp:ListItem>PC</asp:ListItem>
                <asp:ListItem>Xbox</asp:ListItem>
                <asp:ListItem>PS2</asp:ListItem>
                <asp:ListItem>GameCube</asp:ListItem>
            </asp:DropDownList> <br />
            <asp:Button ID="Button3" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Search" /><br />
            <br />
              
            <asp:Label ID="Label2" runat="server" BackColor="Black"
BorderColor="Teal" BorderStyle="Groove"
                ForeColor="Yellow" Text="Game Search"></asp:Label><br />
             
            <asp:TextBox ID="TextBox1" runat="server" BackColor="Lime"
BorderColor="Black" BorderStyle="Outset"
                ForeColor="Black" Width="87px"></asp:TextBox><br />
            <asp:Button ID="Button4" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Search" /><br />
            <br />
            <br />
                
            <asp:Label ID="Label4" runat="server" BackColor="Yellow"
BorderColor="Black" BorderStyle="Groove"
                Text="Genres:"></asp:Label><br />
            <br />
            <asp:Button ID="Action" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
```

```
                Text="Action" /><br />
            <asp:Button ID="Adventure" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Adventure" /><br />
            <asp:Button ID="Fps" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Fps" /><br />
            <asp:Button ID="Puzzle" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Puzzle" /><br />
            <asp:Button ID="Rts" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="Rts" /><br />
            <br />
            <asp:Button ID="Button5" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                Text="View Purchases" Width="110px" /></div>
        <asp:SqlDataSource ID="SqlDataSource1" runat="server"
DataSourceMode="DataReader" ConnectionString="<%$
ConnectionStrings:ConnectionString %>"
            DeleteCommand="DELETE FROM [Table1] WHERE [Game ID] = @Game_ID"
InsertCommand="INSERT INTO [Table1] ([Game Name], [Genre], [System Type],
[System], [Price]) VALUES (@Game_Name, @Genre, @System_Type, @System,
@Price)"
            SelectCommand="SELECT [Game Name], Genre, [System Type], System,
Price, [Game ID] FROM Table1 WHERE ([System Type] = @System_Type)"
UpdateCommand="UPDATE [Table1] SET [Game Name] = @Game_Name, [Genre] =
@Genre, [System Type] = @System_Type, [System] = @System, [Price] = @Price
WHERE [Game ID] = @Game_ID">
            <DeleteParameters>
                <asp:Parameter Name="Game_ID" Type="Int32" />
            </DeleteParameters>
            <UpdateParameters>
                <asp:Parameter Name="Game_Name" Type="String" />
                <asp:Parameter Name="Genre" Type="String" />
                <asp:Parameter Name="System_Type" Type="String" />
                <asp:Parameter Name="System" Type="Boolean" />
                <asp:Parameter Name="Price" Type="Decimal" />
                <asp:Parameter Name="Game_ID" Type="Int32" />
            </UpdateParameters>
            <InsertParameters>
                <asp:Parameter Name="Game_Name" Type="String" />
                <asp:Parameter Name="Genre" Type="String" />
                <asp:Parameter Name="System_Type" Type="String" />
                <asp:Parameter Name="System" Type="Boolean" />
                <asp:Parameter Name="Price" Type="Decimal" />
            </InsertParameters>
            <SelectParameters>
                <asp:Parameter DefaultValue="Xbox 360" Name="System_Type"
Type="String" />
            </SelectParameters>
        </asp:SqlDataSource>
         <asp:GridView ID="GridView1" runat="server"
AutoGenerateColumns="False"
            BackColor="Black" BorderColor="Lime" DataKeyNames="Game ID"
DataSourceID="SqlDataSource1"
            ForeColor="Lime">
```

```
            <RowStyle BackColor="Black" BorderColor="Lime" ForeColor="Lime"
/>
            <EmptyDataRowStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
            <Columns>
                <asp:BoundField DataField="Game Name" HeaderText="Game Name"
SortExpression="Game Name" />
                <asp:BoundField DataField="Genre" HeaderText="Genre"
SortExpression="Genre" />
                <asp:BoundField DataField="System Type" HeaderText="System
Type" SortExpression="System Type" />
                <asp:TemplateField HeaderText="Price" SortExpression="Price">
                    <EditItemTemplate>
                        <asp:TextBox ID="TextBox1" runat="server" Text='<%#
Bind("Price") %>'></asp:TextBox>
                    </EditItemTemplate>
                    <ItemTemplate>
                        $<asp:Label ID="Label1" runat="server" Text='<%#
Bind("Price") %>'></asp:Label>
                    </ItemTemplate>
                </asp:TemplateField>
            </Columns>
            <SelectedRowStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
            <HeaderStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
            <EditRowStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
        </asp:GridView>
        <br />
         <asp:SqlDataSource ID="SqlDataSource2" runat="server"
ConnectionString="<%$ ConnectionStrings:ConnectionString %>"
            DeleteCommand="DELETE FROM [Table3] WHERE [Order ID] = @Order_ID"
InsertCommand="INSERT INTO [Table3] ([User ID], [Price], [Game Name]) VALUES
(@User_ID, @Price, @Game_Name)"
            SelectCommand="SELECT * FROM [Table3]" UpdateCommand="UPDATE
[Table3] SET [User ID] = @User_ID, [Price] = @Price, [Game Name] = @Game_Name
WHERE [Order ID] = @Order_ID">
            <DeleteParameters>
                <asp:Parameter Name="Order_ID" Type="Int32" />
            </DeleteParameters>
            <UpdateParameters>
                <asp:Parameter Name="User_ID" Type="String" />
                <asp:Parameter Name="Price" Type="Decimal" />
                <asp:Parameter Name="Game_Name" Type="String" />
                <asp:Parameter Name="Order_ID" Type="Int32" />
            </UpdateParameters>
            <InsertParameters>
                <asp:Parameter Name="User_ID" Type="String" />
                <asp:Parameter Name="Price" Type="Decimal" />
                <asp:Parameter Name="Game_Name" Type="String" />
            </InsertParameters>
        </asp:SqlDataSource>
        <br />
        <br />
        <asp:CheckBox ID="CheckBox1" runat="server" />
        <img src="../Images/img1.bmp" />
```

```
        <asp:Label ID="Label3" runat="server" ForeColor="Lime"
Text="Assassin's Creed 2 for Xbox 360  Price: $59.99"></asp:Label><br />
        <br />
        <br />
        <asp:CheckBox ID="CheckBox2" runat="server" />
        <img src="../Images/img2.bmp" />
        <asp:Label ID="Label5" runat="server" ForeColor="Lime" Text="Star
Craft II for PC  Price: $59.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox3" runat="server" />
        <img src="../Images/img3.bmp" />
        <asp:Label ID="Label6" runat="server" ForeColor="Lime" Text="Xbox 360
Game System  Price: $199.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox4" runat="server" />
        <img src="../Images/img4.bmp" />
        <asp:Label ID="Label7" runat="server" ForeColor="Lime" Text="Modern
Warfare 2  for Xbox 360  Price: $59.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox5" runat="server" />
        <img src="../Images/img5.bmp" />
        <asp:Label ID="Label8" runat="server" ForeColor="Lime" Text="Modern
Warfare 2  for Ps3  Price: $59.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox6" runat="server" />
        <img src="../Images/img6.bmp" />
        <asp:Label ID="Label9" runat="server" ForeColor="Lime" Text="Super
Mario Brothers for Wii  Price: $49.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox7" runat="server" />
        <img src="../Images/img7.bmp" />
        <asp:Label ID="Label10" runat="server" ForeColor="Lime" Text="Metal
Gear Solid 4 for Ps3  Price:  $59.99"></asp:Label><br />
        <br />
        <asp:CheckBox ID="CheckBox8" runat="server" />
        <img src="../Images/img8.bmp" />
        <asp:Label ID="Label11" runat="server" ForeColor="Lime" Text="Modern
Warfare 2  for PC  Price: $59.99"></asp:Label><br />
        <br />
         <asp:Button ID="Button7" runat="server" BackColor="Black"
BorderColor="Lime"
            ForeColor="Lime" Text="Purchase" /></div>
    </form>
</body>
</html>
```


project2 - Microsoft Visual Studio
File Edit View Project Build Debug Tools Test Window Community Help
Debug
Win32
Solution Explorer - Solution 'project...
main.cpp
IndexedMultiList.cpp
IndexedMultiList.h
main()
C:\windows\system32\cmd.exe
Press any key to continue . . .
Solution E...
Class View
Property ...
Output
Show output from: Build
1>main.cpp
1>Generating Code...
1>Linking...
1>Embedding manifest...
1>Build log was saved at "file://e:\Seagate Sync\VOL\Personal folder\Documents\Senior Portfolio\Coding Projects\Coins 333 Applied Systems\project2\project2\Debug\BuildLog.htm"
1>project2 - 0 error(s), 0 warning(s)
========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ==========
Code Definition Window
Call Browser
Output
Build succeeded
Ln 11
Col 1
Ch 1
INS

```
// Eric Foster
// Project 2
// March 27th, 2009
// March 27th, 2009
// Data Structures
// Grade: 85

#include "IndexedMultiList.h"

IndexedMultiList::~IndexedMultiList(void) {
  Item * cur;
  while (head) {
      cur=head->next;
      delete head;
      head=cur;
  }
}

int IndexedMultiList::size() const {            //returns the total number of
items in the list (using counts)
//If 1 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
//iml.uniqueSize() would return 3 and iml.size() would return 5.
  int total=0;
  for (Item * cur=head; cur; cur=cur->next) total+=cur->count;
  return total;
}

int IndexedMultiList::find(char val) const {    //returns index of position
with val in it or -1 if not found
  int index=0;
  for (Item * cur=head; cur; cur=cur->next, index++) if (cur->data==val)
return index;
  return -1;
}

int IndexedMultiList::countItem (char val) const {    //returns number of
times val is in the list (vacuously 0 if not found)
  for (Item * cur=head; cur; cur=cur->next) if (cur->data==val) return cur-
>count;
  return 0;
}

int IndexedMultiList::totalItemCount (char val) const {
  int total=0;
  for (Item * cur=head; cur; cur=cur->next) if (cur->data == val) total+=cur-
>count;
  return total;
}
int IndexedMultiList::countPosition(int index) const {//returns number of
items at position index (0 based), 0 if bad index
  if (index <0 || index >= used) return 0;
  Item * cur=head;
  for (int i=0; i<index; cur=cur->next, i++) {}
  return cur->count;
}

bool IndexedMultiList::getAt(int index, char &out) const {
```

```
//if position index is in use, out is set to char at that position and true
is returned, otherwise false is returned.
  if (index <0 || index >= used) return false;
  Item * cur=head;
  for (int i=0; i<index; cur=cur->next, i++) {}
  out=cur->data;
  return true;
}

void IndexedMultiList::print() const {//prints out the content of the list in
order by position
  for (Item * cur=head; cur; cur=cur->next) cout << cur->count << " " <<cur-
>data<<endl;
}

//mutators
bool IndexedMultiList::insert(int index, char val, int count) {
/*insert count number of val into the list at position index
returns true iff val was able to fit in list and index was valid and count
>=1
valid index is 0 up to size (inclusive).  So for an empty list, 0 is the only
valid index.
insert should do nothing if index is invalid
If the list as 3 A's in position 1, then 0 and 1 are the only valid indices.
*/
  if (index <0 || index > used || count <= 0) return false;
  if (!head) {
    head=new Item(val, count);
      used++;
      return (head!=NULL);
  }

  Item * cur=head;
  for (int i=0; i<index-1; cur=cur->next, ++i) {}
  Item * newItem=new Item(val, count, cur->next);
  if (!newItem) return false;
  cur->next=newItem;
  used++;
  return true;
}

bool IndexedMultiList::insert(char val, int amount) {
  if (amount <=0) return false;
  if (update(val, amount)) return true;
  return insert(used, val, amount);
}

bool IndexedMultiList::change(int index, int amount) {
/*add amount to item in position index
returns true iff index was valid
insert should do nothing if index is invalid
amount could be negative, if adding amount to the current count results in a
count of <= 0 then
position should be removed
*/
  char out;
  if (getAt(index,out)) return update(out, amount);
```

```
  return false;
}
bool IndexedMultiList::update(char val, int amount) { //finds val in list and
adds amount (which could be negative)
//return true iff found
  for (Item * cur=head; cur; cur=cur->next) {
      if (cur->data==val) {
        cur->count+=amount;
        if (cur->count<=0) removeAll(val);
        return true;
      }
  }
  return false;
}

bool IndexedMultiList::removeByValue(char val) {      //removes 1 instance of
val from list, returns true iff val was in list
  for (Item * cur=head, *prev=0; cur; prev=cur, cur=cur->next) {
      if (cur->data==val) {
        cur->count--;
        if (cur->count==0) {
            if (prev) prev->next = cur->next;
            else head=cur->next;
            delete cur;
            used--;
        }
        return true;
      }
  }
  return false;
}

int IndexedMultiList::removeByIndex(int index) {      //remove position at
index, returns # removed or -1 if index not in use
  if (index <0 || index >=used) return -1;
  char out;
  getAt(index, out);
  int temp=countPosition(index);
  removeAll(out);
  return temp;
}

int IndexedMultiList::removeAll(char val) { //removes all instances of val
from list
        //returns number of items removed, which could be 0
  for (Item * cur=head, *prev=0; cur; prev=cur, cur=cur->next) {
      if (cur->data==val) {
        if (prev) prev->next = cur->next;
        else head=cur->next;
        int count=cur->count;
        delete cur;
        used--;
        return count;
      }
  }
  return 0;
}
```

```
// Eric Foster
// Project 2
// March 27th, 2009
// March 27th, 2009
// Data Structures
// Grade: 85


#pragma once
#include <iostream>
using namespace std;


class IndexedMultiList {
public:
//Constructors/destructors
  IndexedMultiList() : used(0) {head=0;}
  ~IndexedMultiList();  //destructor ONLY if needed

  //accessors
  bool empty() const {return (head==NULL);}     //returns true iff (if and
only if) list is empty
  int uniqueSize() const {return used;}         //returns the number of
positions in the list in use
  int size() const ;          //returns the total number of items in the list
(using counts)
  //If 1 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
  //iml.uniqueSize() would return 3 and iml.size() would return 5.
  int find(char val) const;         //returns index of position with val in
it or -1 if not found
  int countItem (char val) const;   //returns number of times val is in the
list (vacuously 0 if not found)
  int countPosition(int index) const;//returns number of items at position
index (0 based), 0 if bad index
  int totalItemCount (char val) const;
/*returns number of val in the list (vacuously 0 if not found)
Example: there are 3 A's in the first position, 6 B's in the second and 4 A's
in the third position.
find('A') returns 0, find('B') returns 1 and find('C') returns -1
countItem('A') returns 2  (it is in the first and third location)
countItem('B') returns 1, countItem('C') returns 0
totalItemCount('A') returns 7, totalItemCount('B') returns 6,
totalItemCount('C') returns 0
*/
  bool getAt(int index, char &out) const;
  //if position index is in use, out is set to char at that position and true
is returned, otherwise false is returned.
  void print() const;         //prints out the content of the list in order
by position, see below

  //mutators
  bool insert(int index, char val, int count);
  /*insert count number of val into the list at position index
  returns true iff val was able to fit in list and index was valid and count
>=1
  valid index is 0 up to size (inclusive).  So for an empty list, 0 is the
only valid index.
```

```
  insert should do nothing if index is invalid
  If the list as 3 A's in position 1, then 0 and 1 are the only valid
indices.
  */
  bool insert(char val, int amount);//find val in list and add amount, if not
present add to end
  //return true iff already in list
  bool change(int index, int amount);
  /*add amount to item in position index
  returns true iff index was valid
  insert should do nothing if index is invalid
  amount could be negative, if adding amount to the current count results in
a count of <= 0 then
  position should be removed
  */

  bool update(char val, int amount); //finds val in list and adds amount
(which could be negative)
  //return true iff found

  bool removeByValue(char val);     //removes 1 instance of val from list,
returns true iff val was in list

  int removeByIndex(int index);     //remove position at index, returns #
removed or -1 if index not in use

  int removeAll(char val); //removes all instances of val from list
              //returns number of items removed, which could be 0
private:
  struct Item {
      Item (char d, int c, Item *n=0) : data(d), count(c), next(n) {}
      int count;
      char data;
      Item * next;
  };
  int used;
  Item * head;
};
```

```
// Eric Foster
// Project 2
// March 27th, 2009
// March 27th, 2009
// Data Structures
// Grade: 85

#include <iostream>
#include <list>
#include "IndexedMultiList.h"

using namespace std;
```

```
void main() {
  IndexedMultiList iml;
  iml.insert(0, 'D', 4);
  iml.insert(0, 'C', 3);
  iml.insert(1, 'Z', 33);
  iml.insert(3, 'E', 5);
//  iml.print(); cout<<endl;
 // cout <<iml.size() << ", " <<iml.uniqueSize() <<endl;
//  cout <<iml.countNode('Z') << ", " <<iml.countPosition(0) <<endl<<endl;
//  cout <<iml.countNode('a') << ", " <<iml.countPosition(33) <<endl<<endl;

  //iml.remove('C');
  //iml.print();cout<<endl;
  //iml.remove('C');
  //cout << iml.remove('C') <<endl;
  //cout << iml.remove('C') <<endl;
  //iml.print();cout<<endl;
  //iml.update('Z', -33);
  //iml.change(0, -4);
  //iml.print();cout<<endl;
  iml.insert('E', 111);
  iml.insert('F', 222);
  iml.print();cout<<endl;
 /* IndexedMultiList::iterator it;
  it=iml.begin();
  while (it != iml.end()) {
      cout << *it<< ", ";
      it++;
  }*/

}
```

Ethics Papers

1. MS Ban Paper. Written for Applied Systems. Grade: 75.
2. LokiTorrentPaper. Written for Applied Networking. Grade: 80.
3. Adv OS Ethics Paper. Written for Advanced Operating Systems. Grade: 90.

Soft Wars: The Microsoft Word Lawsuit

Eric Foster

Gary Underwood

Coins 333

September 9th, 2009

a change to a completely different word processing program could be worse. For implementing changes in currently available Word products, updates would have to be downloaded but can't guarantee users update to it.

The real issue that needs to be addressed is the process of software patenting. Software patents are usually pretty vague and open the door for lawsuits to occur often. Patents should be required to abide by guidelines which require very specific patenting to avoid a company from easily manipulating a situation to their benefit. While patents attempt to encourage ingenuity, breakthroughs in the software field are now typically generated by million dollar companies who would have to innovate regardless. Software writers should be able to have patents on their specific code but not general ideas to give them a distinct monopoly. Vague patents only strangle out positive innovation for mere greed.

Works Cited:

Keizer, Gregg. "Appeals court grants Microsoft reprieve in Word case." *ARNNET*. 4 Sep 2009. ARN. 9 Sep 2009 <http://www.arnnet.com.au/article/317421/appeals_court_grants_microsoft_reprieve_word_case?fp=39&fpid=27886>.

The New Oxford Annotated Bible. Oxford: Oxford UP, 2007. Print.

Eric Foster

Dr. Lin

Applied Networking

November 23rd, 2009

Privacy of Piracy

The term piracy has changed dramatically in its meaning since the days of pirates at sea. Today the World Wide Web has advanced as a sea which connects all continents but has fallen victim to technological pirates stealing information, media and software. The internet's immense growth has led to advances in everything, including crime. Many pirating and file sharing giants have risen and fallen such as Napster, Pirate Bay and TokiTorrent. New legislation and regulatory agencies seek out the end user in an attempt to punish pirates and inflict serious consequences on violators. The average person knows that stealing is wrong, but when faced with digital media, users find a gray spot and don't see such a clear line of right and wrong anymore. Understanding the effort and work that piracy undermines is important in understanding exactly why it is illegal and wrong.

One of the growth factors contributing to piracy is the lack of consequences and punishment for those involved. Even though several organizations have suffered the consequences, few end violators have been punished compared to the number of violators as a whole. Humans by nature are more inclined to commit a crime if there is almost no punishment imminent for them. Punishing users for internet piracy can be hard when pirating servers and applications don't track their users or downloads. Also determining if a user illegally

downloaded something requires advanced technical personnel as well as advanced resources. Internet Service providers in the U.S. also are no longer required to release user identities as in "December 2003 the U.S. Court of Appeals for the District of Columbia Circuit overturned the earlier rulings, stating that ISPs are not legally obligated to reveal their customers' identities. The ruling does not making file sharing legal, but it seriously hinders the music industry's strategy of targeting individual file sharers."(Torr) This stance makes tracking internet pirates even harder and more costly. France is working on legislation in an attempt to make prosecution quicker and efficient. They have attempted to institute legislation which will give music and film industry associations hire people to trace illegal down loaders and take action. The legislation however has a dramatic loop hole, the ability for one to defend their self when prosecuted. Users could hack into others networks and use it for their own illegal purposes and as a result, the owner of the network would face the punishment, not the actual offender. Further, this requires more on individual users to protect their own networks, ISPs to enforce the law and also open ups policing power to Industry, which could be extremely advantageous or abused. These laws however are considered even worse action by many people. ""The French law will only drive people further underground," Mr. Zimmermann said. "It will make the situation worse.""(O'Brien) As laws and detection advance, so to do the illegal actions, as the criminal always seems one step behind. The number of offenders also grossly outnumbers those fighting piracy, so the odds of success are low.

Another issue concerning Piracy is whether or not people perceive it as illegal. Piracy mainly breaks copyright laws, which some people find to be somewhat ridiculous anyways. The idea is the protection of intellectual property. From Christian ethics, one might find it hard to say that someone owns idea, sound, or sight. Ethics from the bible's stand point are tricky as

such technology didn't exist in ancient times and using that law to govern today's problems is a daunting task. We see movies on TV all the time and we hear music on the radio all the time, somewhat freely. Nothing is stopping us from enjoying free music or movies in this aspect except the ability to hear/see it on demand. Piracy however allows us to do this. Even further many people advocate piracy such as John Perry Barlow, a cofounder of the Electronic Frontier Foundation,[who] has argued that "copyright's not about creation, which will happen anyway—it's about distribution."(Torr) While copyright is supposed to protect creativity and help inhibit innovation, the underlying motivation of greed and profit rears its ugly head and in an ethical sense ruins its definitive appeal of being the ethical cause. Greed, as everyone knows, is the root of all evil. Further in this argument, copyright serves mainly to profit those with the power to fund distribution. The actual artist spends no money but is merely talent and the distribution company is the one taking the endeavor to risk it on them. "Record companies, according to this logic, benefit society by helping to distribute creators' work, and the law should enable them to make a profit in doing so. But, the argument goes, since the Internet has made transmitting information almost free and thus made CDs largely unnecessary as a means of distributing music, record companies are no longer necessary—and neither are the laws that make copying songs illegal."(Torr) In a sense, online media is a "greener" medium of distribution without the need of costly "risks" that current record companies take. The same almost becomes true of movies and software as there are a lot of physical CDs and DVDS created in their distribution. Free distribution of intellectual property also can be interpreted ethically as free education and worldwide collaboration.

In the end to truly decide whether you believe piracy to be wrong, you should ask yourself one question: Would you want others to illegally download your Song or Media and

prevent you from profiting from it? By respecting copyright you also are granted respect by others in being able to copyright your idea or creations. This mutual understanding is vital in all law and government, and so to should be with copyright. The witch-hunt to end piracy also however seems a wasted effort. Anti-piracy attempts merely annoy those who don't pirate or violate copyright laws. People who intend to pirate aren't going to be swayed by shallow threats or attempts to prevent it. Where there's a will there's a way and it's something the industry will have to accept. Rather than spend money on a lost cause, the industry should strive to be the more appealing option.

Works Cited

O'Brien, Kevin. "Plan to Curb Internet Piracy Advances in France." New York Times. 8 April 2009. 22 November 2009. <http://www.nytimes.com/2009/04/09/business/global/09net.html?_r=1>

Torr, James. "Introduction." At Issue: Internet Piracy. Ed. James D. Torr. San Diego: Greenhaven Press, 2005. August 2004. 22 November 2009. <http://www.enotes.com/internet-piracy-article/54083>.

Eric Foster

Gary Underwood

COINS 431

September 14th, 2010

Virtual Violence

Main stream media has portrayed video games as a major cause to violence amongst teens and in schools. Video games however aren't the real problem to blame for mentally unstable children committing foul acts. While many studies may suggest that video games are to blame for violence amongst youth, those researchers are often convicted of false methodology and using skewed findings. Many of the studies attempting to blame video games prove to be inconclusive. Further proving studies wrong, violent crime amongst youth is at a thirty year low (Jenkins). Video games provide a way for many to vent in a "play" environment and to avoid physical alternatives. Video games also often challenge gamers and significantly increase hand-eye coordination. Video games serve as a form of expression and have freedom through the first amendment. Violence in video games may be somewhat disturbing at times, but they aren't real acts which would be deemed unethical. Freedom of violence in a video game is no difference than the freedom to imagine it in your own mind. It is important however to avoid selling inappropriate games to minors without proper parent judgment or consent. In the end it is up to parents to prevent their children from playing material they deem unacceptable.

Video games are rated by the ESRB, Entertainment Software Rating Board, with ratings ranging from Early Childhood to Adults only. Adults only ratings suggest that content only be

viewed by individuals of 18 or older as where Mature ratings suggest 17 or older. The ESRB rating system even includes descriptors to accurately describe the content which decide the rating. The descriptors include things such as partial nudity, violence, and gambling (ESRB). Currently no law requires video games to be rated nor requires vendors to avoid selling mature rated games to minors without a parent or guardian present. Most retailers voluntarily require ID to purchase a mature rated game. The biggest change that needs to happen is a law or act to make ID checks mandatory to make sure that children don't acquire potentially disturbing content without a parent or guardian's approval.

Violence in video games isn't going to stop anymore than violence on Television. Violence in video games sells so developers have no reason not to implement what sells. If parents want to prevent their children from being exposed to violence, they need to better monitor their children and material the children observe. Retailers however can be the more cautious and enact preventative measures to keep children from acquiring games without proper parental consent. Most retailers voluntarily require proof of age to buy mature or adult games, but not all do as they aren't required to do so by law. Without punishment, how can they truly be forced to abide? The main question is whether or not youths are being able to buy games on their own, or if parents are blindly buying games for children and ignoring ESRB ratings that clearly outline the game's age content. In 2008 the Federal Trade Commission conducted undercover shopping to figure out what percentage of mature and adult rated games were being sold to youths without requiring proof of age or parental consent. The teens used in the undercover shopping survey were ages 13-16, not 17 or 18, as mature and adult games are recommended for by the ESRB rating system. The FTC found that about 20% of teens managed to succeed in their purchase, but that means 80% were prevented. An 80% success rate is fairly

good considering it isn't even required for retailers to prevent. However, 80% still isn't high enough (FTC). More should be done to stop kids from purchasing inappropriate games without parental consent.

In Exodus 19:20 Moses introduces the Ten Commandments and with it "Honor thy Mother and Father". Just as we should honor our parents and not purchase material that they deem unsuitable, vendors should honor all parents by interceding where parents aren't present. Vendors are the point of sales to children and can be the ones to help prevent sales. Sure children may acquire games from friends or have friend purchase games for them, but parents are actually the culprit of most underage game sales. According to Dr. Henry Jenkins, the FTC found that 83% of underage games sales actually were by the parents or parents with their children (Jenkins). Really violence in video games isn't to blame for influencing children. Parents themselves are guilty for the purchase of games without proper discretion. Vendors can again intercede when parents purchase games for children, or with their children present to inform them of the rating of the game and what descriptors are used to describe the game content. By engaging parents and making sure they are aware of the rating and content, vendors can honor parent wishes and help prevent underage game sales. If a parent still continues to make the purchase after the vendor has mentioned its content, then only the parent can be to blame for making a bad decision.

Another argument against video games is that because they are used to train soldiers, then we are training our children in the same way. Dr. Henry Jenkins does a great job in debunking this myth. Jenkins makes the point that to believe that, we disregard that learners have a conscious or have any resistance to what they are taught. Assuming parents have taught their kids right and wrong, as the Bible should instill in them, then children will be able to discern

from the wrongs in video games and what is right in life application. Also Jenkins claims that we must also assume they apply what they learn in the video game to their everyday life. Video games serve to challenge players and test their abilities. Violence may seem to be an unnecessary element in games, but violence is a part of life that we would be naïve to ignore. Games have their own rules to which the gamer must abide by to actually succeed in the game. Most violent games have penalties in the event gamers use bad judgment or random violence without a point. Games don't create killers; bad environments do (Jenkins).

Works Cited

Fentonmiller, Keith. "Undercover Shoppers Find It Increasingly Difficult for Children to Buy M-Rated Games". Federal Trade Commission. May 8th, 2008. September 12th, 2010. http://www.ftc.gov/opa/2008/05/secretshop.shtm.

"Game Ratings and Descriptor Guide". Entertainment Software Rating Board. September 12th, 2010. http://www.esrb.org/ratings/ratings_guide.jsp.

Jenkins, Henry. "Eight Myths About Video Games Debunked". PBS. September 12th, 2010. http://www.pbs.org/kcts/videogamerevolution/impact/myths.html.

Presentations

1. Android Restaurant Application – Written for Senior Project. Explains my intentions for my senior project using an android application. Grade: 100.
2. XFS – Written for Advanced Operating Systems. Explains the XFS file system used in Linux. Grade: 100.

Eric Foster
November 9th, 2010
Advanced Operating Systems

XFS : A HIGH-PERFORMANCE JOURNALING FILE SYSTEM BY SILICON GRAPHICS

XFS: ORIGIN

- Originally created for use with Silicon Graphics IRIX operating system.
- Later ported to the Linux Kernel.

WHAT IS A JOURNALING FILESYSTEM?

- A file system that keeps a log to track all changes it makes or attempts to make.
- Saves to a serial log before even actually saving to the disk
- Allows fast recovery upon crash or interruption.

WHY USE XFS?

- Its forte is its handling of large files.
- 64 bit file system, so very high capacity.
- Great for multi-core systems due to Allocation groups.
- Striped Allocation
- Extent Based allocation.
- Variable Block Sizes
- Delayed Allocation

ALLOCATION GROUPS

- The XFS filesystem is partitioned into allocation groups. All are equally sized. Multiple files and Directories can span multiple allocation groups.
- Each group manages its own freespace and "Inodes".
- Inodes store information about objects in the file system, IE: Files and directories.
- Allow for parallelism, great for multi-cores and multi-threads.

STRIPED ALLOCATED

- If you create a striped RAID set up, you can define a stripe when you create the file system. Allows max performance, by making sure that data and Inode allocations are aligned with your stripe.

VARIABLE BLOCK SIZES

- XFS allows block sizes to range between 512 bytes and 64 kilobytes which allows the file system to be fine tuned to the intended use.

DELAYED ALLOCATION

- Uses a technique in which block allocation doesn't occur until the data is fully flushed to disk.
- Helps maintain contiguous blocks, and helping reduce fragmentation problems.

DIRECT I/O

- Allows non-cached I/O directly to a userspace. Uses DMA to transfer data between the application buffer and the disk.
- Provides the full I/O bandwidth of the disk devices.

Eric Foster
November 3rd, 2010
Senior Project

ANDROID RESTAURANT APPLICATION

Old Project Idea Rethought

- Was going to grab menus off of each restaurant site using a search algorithm.
- Every site is too different and any algorithm to accurately find restaurants and extract data could be too hard.
- Google's Application to get restaurant menus just pulls from one site, www.allmenus.com

So no special work there...

New Idea

- Simply redesign a restaurant search tool and add unique features to provide a better application.
- Add a search for a specific food within a specific area. IE: You want chicken wings, so you type in chicken wings.
- Match against existing menus to locate restaurants providing the desired food within so many miles of your current location.

Some Issues

- Slang or special lingos for food need to be taken into account. A lot of places have their own unique names for their food.
- Searching for "fried mushrooms". Will also need to know that "fried shrooms" could also be a result.

Other Features

- Other features will of course need to be added to make the application sophisticated and worth using over another application.
- Add an online order feature for restaurants supporting online orders.

Current Goals

- Get application finding locations via GPS.
- Get application to determine distance between Android device and locations.
- Display maps via the Google Maps API.
- Develop and Add Unique Features.
- Develop a user friendly and aesthetically pleasing UI.
- Add built-in Gratuity Calculator where applicable.